



BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

- P – Performance Measure (how the agent's success is evaluated)
- E – Environment (the world in which the agent operates)
- A – Actuators (how the agent performs actions)
- S – Sensors (how the agent perceives the environment)



2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width`, `self.height`, `self.walls`, `self.food_positions`, `self.opponents`, and `self.agent_pos` represent the environment state. They define the grid dimensions, obstacles, food locations, opponent locations, and the agent's position in the environment.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This is a Multi-Agent environment because, in addition to the main agent, the code creates opponents using `self.opponents`. These opponents move independently and interact with the agent, making the environment involve multiple agents.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all percepts received by an agent through its sensors from the beginning of its operation until the current time. It is used by the agent to make decisions about future actions.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method represents the Sensors (S) component of the PEAS framework because it collects and returns information about the current state of the environment, including the agent's position, opponent positions, food detection, wall detection, collision status, score, and remaining food. This percept information helps the agent decide its next action.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The environment is Partially Observable because the `get_percept()` method does not provide the complete state of the environment. It only returns the agent's current position, opponent positions, whether it is on a food cell or wall, collision status, score, and the number of remaining food items. The agent cannot see the locations of all food items or all walls, so it does not have complete knowledge of the environment.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure (P) is being updated because hitting a wall reduces the agent's score by 5 points (`self.score -= 5`). The score measures how well the agent is performing, so changing it updates the performance measure.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important because the performance measure should evaluate the results of the agent's actions in the external environment, not its internal processing. In this code, the score decreases when the agent hits a wall because hitting a wall is an undesirable environmental outcome. The agent is evaluated based on how well it achieves its goals (collecting food and avoiding obstacles), rather than how much computation or effort it uses internally.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hiding `self.toxic_traps` from the `get_percept()` method makes the environment partially observable because the agent cannot perceive the locations of the toxic traps. Since important information about the environment is hidden, the agent does not have complete knowledge of its surroundings and must make decisions with incomplete information.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in`

`self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'` , you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding `smells_toxin` expands the agent's percept sequence by providing additional information about toxic traps. This helps the agent act more rationally because it can use this new percept to avoid toxic traps and choose actions that maximize its expected utility (performance score). A rational agent makes decisions based on the information available from its percept sequence, so having an additional sensor enables better decision-making.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World exploit was that an agent could maximize its performance measure by repeatedly making the environment dirty and then cleaning it again, instead of simply keeping the environment clean. This demonstrated that poorly designed performance measures can encourage unintended or exploitative behavior.

